

Module 6 Week B — Applied Lab: Embeddings Comparison

Build TF-IDF (Term Frequency–Inverse Document Frequency), GloVe (Global Vectors for word representation), and DistilBERT (a distilled version of BERT) representations for the same corpus and compare how each method ranks text similarity. You will implement three different text-to-vector pipelines and discover where they agree, where they disagree, and why.

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

Learning Objectives

After completing this lab, you will be able to:

- Build TF-IDF representations for a text corpus using `TfidfVectorizer`
- Load pre-trained GloVe vectors and compute average document embeddings
- Extract contextual embeddings from DistilBERT using the tokenizer-model-pooling pipeline
- Compute pairwise cosine similarity matrices for each representation
- Compare similarity rankings across methods and analyze where they diverge

Setup

1. [Accept the assignment](#)
2. Clone your repo and open it in VS Code.
3. The repo contains:
 - `data/bbc_news.csv` – ~1,000 BBC News article snippets balanced across 5 categories (200 per category: business, entertainment, politics, sport, tech). Lab 6B uses a categorized news corpus rather than the climate articles you saw in Week A so that the three-way method comparison surfaces a richer set of agreements and disagreements; the Integration Task in Week B continues with the climate corpus.
 - `data/glove_50k_50d.txt` – pre-extracted GloVe embeddings (50,000 words, 50 dimensions)

4. **Install dependencies:**

```
pip install -r requirements.txt
```

`transformers` requires PyTorch for the DistilBERT tasks (Tasks 3-4). `torch` is listed in `requirements.txt` (version 2.2 or higher), and `pip install` will download the correct version for your computer automatically. Intel Macs need 2.2.x because PyTorch dropped x86_64 macOS wheels at 2.4; Apple Silicon and Linux learners pick up a newer release. The first time you import the `transformers` library and load `distilbert-base-uncased`, the model files (~250 MB) will be downloaded and cached (saved) on your computer. After the first time, the model loads from the cache on your computer.

5. **Create a working branch:**

```
git checkout -b lab-6b-embeddings
```

Tasks

Complete the implementation of `embeddings_lab.py`. The file contains function signatures with docstrings – you implement the logic.

Task 1 – Build TF-IDF Representations

Implement `build_tfidf(texts)`:

- Use `TfidfVectorizer` from scikit-learn to build TF-IDF representations for all texts
- Return a tuple `(tfidf_matrix, vectorizer)` where `tfidf_matrix` is the sparse matrix (shape: number of texts by vocabulary size) and `vectorizer` is the fitted `TfidfVectorizer` instance — the vectorizer is needed downstream so the `_main` block unpacks both:
`tfidf_matrix, vectorizer = build_tfidf(texts)`

Then implement `compute_tfidf_similarity(tfidf_matrix)`:

- Compute the pairwise cosine similarity matrix using `cosine_similarity` from `sklearn.metrics.pairwise`
- Return the similarity matrix (shape: n by n, values in [0, 1], diagonal approximately 1.0)

Task 2 – Build GloVe Average Embeddings

Implement `load_glove(filepath)`:

- Load the GloVe file into a dictionary mapping words to numpy arrays (same approach as the drill)

Then implement `text_to_glove(text, embeddings)`:

- Split the text into words (lowercase)
- Look up each word in the embeddings dictionary
- Skip words not in the vocabulary (OOV words)
- Return the average of all found word vectors as a numpy array of shape `(50,)`
- If all words are OOV, return a zero vector of shape `(50,)`

The output dimension is fixed at 50 because the provided GloVe file (`data/glove_50k_50d.txt`) is 50-dimensional — there is no `dim` parameter in the signature.

Use these two functions to compute GloVe embeddings for all texts, then compute the pairwise cosine similarity matrix.

Task 3 – Extract DistilBERT Embeddings

Implement `extract_bert_embedding(text, tokenizer, model)`:

- Tokenize the text with `truncation=True` and `max_length=512`
- Run the forward pass with `torch.no_grad()`
- Extract `last_hidden_state` from the model output
- Apply mean pooling over the token dimension (accounting for the attention mask)
- Return the resulting embedding as a numpy array of shape `(768,)`

Load `distilbert-base-uncased` using `AutoTokenizer` and `AutoModel` from `transformers`. Set the model to evaluation mode with `model.eval()`.

Compute DistilBERT embeddings for all texts, then compute the pairwise cosine similarity matrix.

Task 4 – Compare Similarity Rankings

Implement `compare_similarities(texts, queries, tfidf_sim, glove_embeddings, bert_model, bert_tokenizer)`:

- `texts` is the full list of corpus strings; `queries` is the list of query strings — the `_main` block selects the first article from each of the 5 categories so the cross-method comparison spans the full corpus rather than a single category
- `tfidf_sim` is the precomputed TF-IDF similarity matrix from Task 1
- `glove_embeddings` and `bert_model / bert_tokenizer` are passed in raw (not pre-computed similarity matrices) — your function recomputes per-query GloVe and BERT similarity on the fly
- For each query string, find the top-3 most similar texts according to each method, excluding the query itself
- Return a dictionary keyed by **query string** (not index), with the structure:

```
{
  query_text: {
    "tfidf": [(text, score), ...], # top-3
    "glove": [(text, score), ...], # top-3
    "bert":  [(text, score), ...], # top-3
  }
}
```

In `_main`, print a comparison table for each query showing the top-3 results from each method side by side.

Task 5 – Analyze

In your PR description, answer these questions:

1. The corpus has 5 categories (business, entertainment, politics, sport, tech). For which queries do all three methods return top-3 results from the **same** category? When the methods agree on category, do they also agree on the specific articles?
2. Identify at least one query where TF-IDF and DistilBERT disagree — for example, TF-IDF stays within the query's category while DistilBERT reaches across categories, or vice versa. What does this tell you about keyword overlap versus semantic meaning?
3. Compute the OOV rate for your corpus against the 50,000-word GloVe vocabulary, and describe what kinds of words are missing. (Proper nouns? Domain-specific terminology? Numbers? Punctuation artifacts?) How might these missing words shape the GloVe averages on this corpus?

Running Tests Locally

Run the autograder tests with:

```
pytest tests/ -v
```

Note on BERT tests: This assignment uses PyTorch and Hugging Face transformers for the DistilBERT embedding tasks (Tasks 3-4). `torch` is listed in `requirements.txt`, so all tests will run locally after `pip install -r requirements.txt`.

If you see 2 tests marked as **skipped** (the BERT-related tests), it means torch is not in your current environment. The non-BERT tests (Tasks 1-2, Task 5) will still run and give you feedback on TF-IDF, GloVe, and comparison logic. To get full local coverage, run:

```
pip install -r requirements.txt
```

On CI (when you push), all tests always run — the autograder workflow installs the same `requirements.txt` before running pytest.

Submission

Open a pull request from `lab-6b-embeddings` to `main`. Your PR must include:

1. Completed `embeddings_lab.py` that runs end-to-end
2. PR description with:
 - Comparison table for 5 queries: top-3 similar texts per method
 - Analysis of where methods agree and disagree
 - OOV rate observation
3. Paste your PR URL into TalentLMS -> Module 6 Week B -> Lab 6B to submit this assignment.

Challenge Extensions

These are optional extensions for learners who finish the base requirements early. They are not required for credit but will deepen your understanding. Each tier builds in complexity.

Tier 1 – Embedding Normalization and Its Effect

Cosine similarity implicitly normalizes vectors, but the raw vector magnitudes carry information too.

- Compute L2-normalized embeddings for all three methods (divide each vector by its norm)
- Compare Euclidean distance rankings to cosine similarity rankings for the same 5 queries
- Identify a case where normalization changes the ranking order
- Write a brief note: when does the distinction between cosine similarity and Euclidean distance on normalized vectors matter? When does it not?

Tier 2 – Subword Tokenization Analysis

BERT's WordPiece tokenizer handles OOV words by splitting them into known subword pieces – a fundamentally different approach from GloVe's "skip it" strategy.

- For each text in the corpus, compare the tokenization from a simple whitespace split (used for GloVe) versus the BERT tokenizer
- Compute the average number of subword tokens per whitespace word for each text
- Identify the 10 words most frequently split into subwords by the BERT tokenizer
- Analyze: for domain-specific vocabulary (climate science terms), does subword tokenization produce meaningful pieces or arbitrary fragments?
- Write a paragraph comparing the two OOV strategies and their implications for embedding quality on specialized text

Tier 3 – Build a Retrieval Evaluation Harness

The base task compares similarity rankings qualitatively. A production system needs quantitative evaluation.

- Define 10 query-document relevance pairs for the climate corpus (manually label which documents are relevant to each query)
- Implement Mean Reciprocal Rank (MRR) and Precision@K (P@3, P@5) for each representation method
- Build a function that takes a list of queries with relevance labels and produces a comparison table of retrieval metrics across all three methods
- Run the evaluation and write a structured analysis: which method is best for this corpus and these queries? Would you expect the same ranking on a different domain (e.g., product reviews, legal documents)?
- Design (but do not implement) a hybrid retrieval approach that uses TF-IDF for initial candidate selection and BERT for re-ranking. What are the computational tradeoffs?

